GUITARDS: MODELADO COMPUTACIONAL DEL MÁSTIL DE GUITARRA MEDIANTE ESTRUCTURAS DE DATOS

Samuel Stiven Escobar Rodriguez, Cristian Daniel Silva Rodriguez
Universidad Nacional de Colombia
Sede Bogotá
Bogotá D.C., Colombia
{sescobarro, crsilva} @unal.edu.co

Resumen—El análisis de notas, escalas y acordes en la guitarra requiere relacionar posiciones físicas del mástil con conceptos musicales abstractos, lo que dificulta su estudio y su procesamiento computacional. Este proyecto propone un sistema que modela el mástil como una estructura de datos y permite consultar notas e intervalos de forma eficiente. La solución representa el mástil como una matriz bidimensional de notas precomputadas a partir de la afinación, utilizando aritmética modular para la transposición. Se emplean arrays, listas, diccionarios y conceptos de listas circulares, seleccionados técnicamente según las operaciones requeridas. En esta primera entrega se implementa el modelo del dominio, el cálculo de notas por posición en tiempo constante y la consulta de intervalos.

Index Terms—estructuras de datos, guitarra, matriz bidimensional, aritmética modular, modelado de dominio.

I. INTRODUCCIÓN

La guitarra es un instrumento en el que una misma nota musical puede interpretarse en múltiples posiciones físicas del mástil, y donde las relaciones entre notas (intervalos, escalas, acordes) siguen reglas matemáticas precisas. Para un estudiante o intérprete, comprender qué nota suena en cada combinación de cuerda y traste, así como los intervalos entre ellas, requiere un esfuerzo cognitivo considerable cuando se realiza de forma manual.

La situación específica que motiva este proyecto es la necesidad de contar con una herramienta computacional que no solo muestre información musical, sino que estructure el mástil de la guitarra como un ente procesable. Las aplicaciones comerciales existentes suelen enfocarse en la interfaz gráfica o en la enseñanza guiada, pero no exponen de manera abierta el modelo de datos subyacente ni permiten consultas algorítmicas sobre él. Por otro lado, los libros y tablaturas tradicionales presentan la información de forma estática.

La relevancia de este problema radica en que el mástil de la guitarra es, en esencia, un grafo o una matriz con reglas de transición claras. Modelarlo computacionalmente permite no solo resolver consultas inmediatas (como identificar una nota), sino también sentar las bases para resolver problemas complejos en el futuro, como la generación automática de digitaciones de acordes o la búsqueda de patrones de escalas sobre el diapasón.

La propuesta presentada en este trabajo consiste en diseñar e implementar un sistema en Java que modele el mástil como una matriz bidimensional precomputada. El sistema utiliza aritmética modular para calcular la transposición de notas (tratando las 12 notas cromáticas como una lista circular) y almacena la información en estructuras de datos adecuadas para garantizar consultas en tiempo O(1).

El resto del documento se organiza de la siguiente manera: la Sección II presenta el trabajo relacionado; la Sección III detalla los objetivos; la Sección IV define el alcance y las limitaciones; la Sección V los requerimientos; la Sección VI describe el diseño de la solución; la Sección VII justifica las estructuras de datos seleccionadas; la

Sección VIII reporta el estado de la implementación; y finalmente, la Sección IX presenta las conclusiones preliminares.

II. TRABAJO RELACIONADO

En el ámbito del software musical, existen diversas herramientas que abordan la teoría musical y la representación de instrumentos. Librerías como Tonal.js [1] o music21 [2] permiten manipular notas, intervalos y acordes de manera programática. Sin embargo, su enfoque principal es la teoría musical abstracta (el "qué" suena), prestando poca atención a la representación física del instrumento (el "dónde" y "cómo" se ejecuta).

Por otro lado, aplicaciones comerciales de aprendizaje como Yousician o Fender Play ofrecen una representación visual del mástil, pero operan como cajas negras: su modelo de datos es propietario y no permiten a los usuarios realizar consultas personalizadas o integrar la información en sus propios desarrollos.

El vacío que justifica esta propuesta es la falta de un modelo de datos abierto, técnicamente justificado y eficiente que una la representación física del mástil (cuerdas y trastes) con la teoría musical subyacente, sirviendo como motor para futuros sistemas de análisis o recomendación. La propuesta se diferencia al priorizar la estructura de datos y la eficiencia algorítmica sobre la interfaz gráfica.

III. OBJETIVOS

Objetivo general: Diseñar e implementar un sistema en Java que modele el mástil de guitarra como una estructura de datos, permitiendo consultar notas e intervalos de forma eficiente, como base para el análisis futuro de escalas y acordes.

Objetivos específicos:

1. Representar el mástil como una matriz de notas precomputadas a partir de la afinación estándar.
2. Implementar la transposición de notas mediante aritmética modular, tratando la escala cromática como una lista circular.
3. Calcular el intervalo en semitonos entre dos notas dadas y determinar su nombre técnico.
4. Consultar la nota en cualquier posición del mástil (cuerda, traste) con una complejidad temporal de O(1).
5. [Planificado] Generar escalas y acordes como conjuntos de notas y buscar sus posiciones en el mástil en futuras entregas.

IV. ALCANCE Y LIMITACIONES

Para garantizar la viabilidad del proyecto en el tiempo estipulado, se establecen los siguientes límites:

- **Funcionalidades desarrolladas en esta entrega:** Modelado de la afinación estándar (EADGBE), construcción de la matriz del mástil (6 cuerdas por N trastes configurables), cálculo de la nota para cualquier par (cuerda, traste), y cálculo de intervalos en semitonos y su nombre técnico.
- **Funcionalidades fuera del alcance (entregas futuras):** Generación de acordes y sus respectivas digitaciones (Entrega 2), generación de escalas y modos musicales mediante conjuntos (Entrega 2), búsqueda de caminos en el mástil mediante BFS/DFS y organización de escalas mediante árboles (Entrega 3).
- **Restricciones técnicas:** El desarrollo se realiza en el lenguaje Java sin dependencias de librerías externas para la lógica central. La interfaz actual es de línea de comandos (consola).

- **Supuestos:** Se asume una afinación estándar EADGBE. Las notas se representan internamente como enteros (0-11) para facilitar la aritmética modular. El mástil se construye una vez (precomputación) y se asume que será consultado múltiples veces, justificando esta decisión de diseño.
- **Limitaciones:** En esta primera versión, se utiliza únicamente la notación con sostenidos (C#, D#, etc.) para simplificar la representación, dejando la enarmonía (por ejemplo, C# vs Db) y la notación con bemoles para etapas posteriores.

V. REQUERIMIENTOS PRINCIPALES

Los requerimientos funcionales que el sistema aborda en su primera versión son los siguientes:

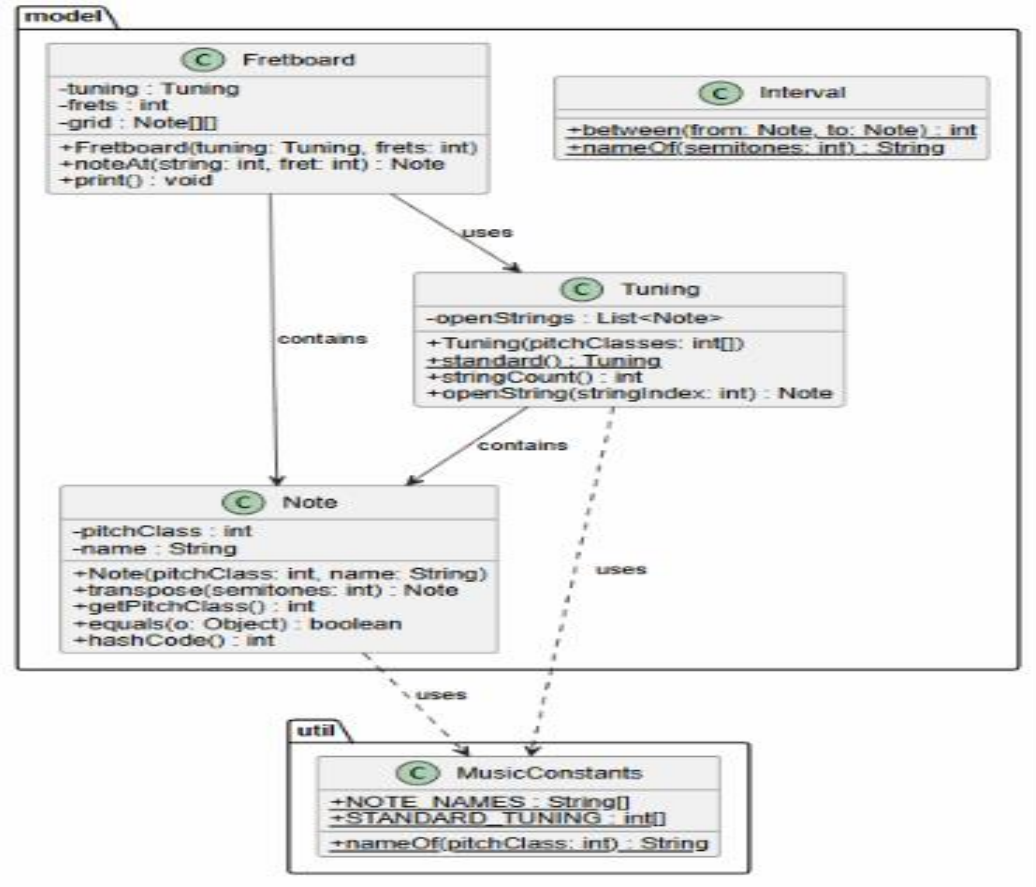
- **RF1:** El sistema debe construir el modelo del mástil a partir de una afinación (lista de notas al aire).
- **RF2:** Dado un número de cuerda y un número de traste, el sistema debe devolver la nota musical correspondiente.
- **RF3:** El sistema debe calcular el intervalo en semitonos entre dos notas musicales dadas.
- **RF4:** El sistema debe devolver el nombre técnico de un intervalo a partir de su cantidad de semitonos.
- **RF5:** El sistema debe ser capaz de imprimir el mástil completo de forma legible para permitir su verificación visual

Requerimientos no funcionales:

- **RNF1:** La operación de consulta de nota por posición (RF2) debe ejecutarse en tiempo constante O(1).
- **RNF2:** El código debe seguir una arquitectura de paquetes que separe el modelo de dominio de las utilidades de configuración.

VI. DISEÑO DE LA SOLUCIÓN

La solución se ha diseñado siguiendo un enfoque orientado a objetos, dividiendo las responsabilidades en dos paquetes principales: util y model. La dirección de las dependencias fluye desde model hacia util; es decir, las clases del modelo utilizan las constantes globales, pero las utilidades no conocen a las clases del modelo, evitando dependencias circulares.



Flujo de información y funcionamiento:
El sistema inicia con la carga de la afinación desde MusicConstants (un array de enteros que representa los pitch classes). La clase Tuning toma este array y construye una lista de objetos Note, que son inmutables. Posteriormente, la clase Fretboard toma el objeto Tuning y precomputa una matriz bidimensional de notas aplicando la transposición. Finalmente, la clase Interval permite calcular la distancia en semitonos entre dos instancias de Note.

Pseudocódigo del algoritmo de construcción del mástil:

```
ALGORITMO ConstruirMastil(afinacion, numeroTrastes)
  PARA cada cuerda s DESDE 0 HASTA (afinacion.tamaño - 1) HACER
    notaAlAire = afinacion.obtenerNota(s)
    PARA cada traste f DESDE 0 HASTA numeroTrastes HACER
      matriz[s][f] = notaAlAire.transponer(f)
    FIN PARA
  FIN PARA
FIN ALGORITMO
```

Ejemplo de funcionamiento:
Si el sistema recibe la consulta noteAt(0, 5) (cuerda 6, traste 5), el sistema accede a la matriz en la posición [0][5] y retorna la nota A (La). Si posteriormente se solicita calcular el intervalo entre la nota A y la nota C, el sistema retorna 3 semitonos y su nombre técnico: "Tercera menor".

VII. ESTRUCTURAS DE DATOS SELECCIONADAS

La selección de estructuras de datos se realizó basándose estrictamente en la naturaleza de la información y las operaciones requeridas para cada componente, evitando el uso artificial de estructuras.

A. Array bidimensional (Matriz)

- **Información que representa:** Las notas musicales en cada posición física del mástil (cuerda x traste).
- **Operaciones:** Acceso directo mediante índices [cuerda][traste].
- **Justificación:** El acceso por coordenadas es la operación dominante. Una matriz proporciona acceso en tiempo O(1) con una sobrecarga de memoria mínima.
- **Alternativas:** Se consideró una List<List<Note>>, pero una matriz ofrece una estructura más rígida y algorítmicamente más rápida para dimensiones fijas conocidas.
- **Costos:** Acceso O(1). Construcción O(C × T), donde C es el número de cuerdas y T el número de trastes.

B. Lista (ArrayList)

- **Información que representa:** La secuencia ordenada de notas al aire de las cuerdas (de la sexta a la primera).
- **Operaciones:** Inserción al final (add), acceso por índice (get), y consulta de tamaño (size).
- **Justificación:** Se requiere preservar el orden de las cuerdas. La clase ArrayList de Java ofrece un tamaño dinámico y un acceso indexado O(1), además de una API rica para futuras expansiones.
- **Alternativas:** Un array simple Note[] es viable y ligeramente más eficiente en memoria, pero List desacopla la implementación y facilita la manipulación en código.
- **Costos:** Acceso O(1), Inserción al final O(1) amortizado.

C. Diccionario (HashMap)

- **Información que representa:** La relación entre una cantidad de semitonos y el nombre técnico del intervalo musical (ej. 4 -> "Tercera mayor").
- **Operaciones:** Búsqueda por clave (get).
- **Justificación:** Aunque los semitonos (0-11) son claves contiguas y un array sería técnicamente válido, se optó por HashMap por su claridad semántica (mapea un concepto a otro) y para facilitar la extensión futura a intervalos compuestos (ej. 12 semitonos = octava) sin necesidad de redimensionar un array.
- **Costos:** Búsqueda O(1) en promedio.

D. Lista circular (Concepto algorítmico)

- **Información que representa:** La naturaleza cíclica de las 12 notas cromáticas.
- **Operaciones:** Transposición (avanzar o retroceder semitonos).
- **Justificación:** Se implementa mediante aritmética modular (usando Math.floorMod para manejar correctamente valores negativos). Esto permite que si se transpone la nota B (11) un semitono, el sistema retorne C (0) sin desbordar los límites lógicos.
- **Costos:** O(1).

E. Decisión transversal: Igualdad e identidad (equals y hashCode)

La clase Note es inmutable y su identidad está definida únicamente por su pitchClass (entero 0-11), ignorando su atributo de texto (nombre). La correcta implementación de equals y hashCode basada en el pitchClass es fundamental para el futuro uso de tablas hash (Sets y Maps) en la Fase 2, garantizando que dos notas como C# y Db, si compartieran la misma frecuencia, sean tratadas como el mismo elemento en un conjunto, optimizando la búsqueda de pertenencia a O(1).

VIII. ESTADO DE LA IMPLEMENTACIÓN

La implementación se encuentra alojada en un repositorio de GitHub. El estado de los componentes desarrollados hasta la fecha es el siguiente:

- **Implementado:**
  - Clase Note: Representación inmutable de la nota con transposición modular.
  - Clase MusicConstants: Tablas de configuración de notas y afinación estándar.

- Clase Tuning: Construcción de las notas al aire.
- Clase Fretboard: Creación de la matriz precomputada, consulta O(1) e impresión del mástil.
- Clase Interval: Cálculo de distancias y nombres de intervalos.
- Clase Main: Punto de entrada con una demostración funcional.

- **Parcialmente implementado / Pendiente:**
  - Búsqueda de posiciones de una nota en el mástil (findAll).
  - Generación de escalas (Set de notas).
  - Acordes y digitaciones.
  - Búsqueda en grafo (BFS/DFS) sobre el mástil.
  - Implementación de la enarmonía.

IX. CONCLUSIONES PRELIMINARES

La primera fase del proyecto establece un modelo de dominio robusto para el mástil de la guitarra. Se concluye que el uso de una matriz precomputada es la estrategia óptima para las consultas de posición, garantizando un tiempo O(1) que será crítico en las fases posteriores cuando se requiera evaluar miles de posiciones para encontrar patrones de escalas o acordes.

Asimismo, la inmutabilidad de la clase Note y la definición de su identidad mediante el pitchClass preparan el sistema para el uso de conjuntos (Sets), lo que permitirá en la próxima entrega modelar escalas y acordes de manera matemáticamente elegante y computacionalmente eficiente.

REFERENCIAS

[1] D. Tarr, "Tonal.js: A functional music theory library," GitHub, 2024. [En línea]. Disponible: https://github.com/tonaljs/tonal

[2] M. S. Cuthbert y C. Ariza, "music21: A Toolkit for Computer-Aided Musicology," MIT, 2024. [En línea]. Disponible: https://web.mit.edu/music21/